
]

RAPPORT DE STAGE D'IMMERSION INDUSTRIELLE

Filière : Génie Informatique — 4^{ème} Année

Conception et Développement d'un Prototypé de Jumeau Numérique Industriel

Pour la Supervision Temps Réel, la Détection d'Anomalies et l'Analyse d'Impact d'une
Chaîne de Production de Phosphate

Auteur : [NOM ET PRÉNOM DE L'ÉTUDIANT]
Élève Ingénieur en Génie Informatique (4A)

Encadrant Professionnel : [ENCADRANT PROFESSIONNEL]
Ingénieur / Chef de Projet — Groupe OCP

Encadrant Pédagogique : [ENCADRANT PÉDAGOGIQUE]
Enseignant-Chercheur — ENSA Safi

Remerciements

Au terme de ce stage d'ingénieur d'une durée de deux mois réalisé au sein du Groupe OCP, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué au bon déroulement de cette expérience professionnelle.

J'adresse mes vifs remerciements à la direction du Groupe OCP pour m'avoir accueilli dans un environnement industriel de premier plan mondial et pour la mise à disposition des moyens techniques nécessaires à l'accomplissement de cette mission.

J'exprime ma sincère reconnaissance à mon encadrant professionnel, [ENCADRANT PROFESSIONNEL], pour sa confiance, son encadrement bienveillant, sa disponibilité constante et la pertinence de ses conseils techniques tout au long du développement de ce projet de jumeau numérique.

Mes remerciements s'adressent également à mon encadrant pédagogique à l'ENSA Safi, [ENCADRANT PÉDAGOGIQUE], pour son suivi rigoureux, ses exigences méthodologiques et ses orientations scientifiques précieuses.

Je remercie chaleureusement l'équipe technique et opérationnelle du département d'accueil pour son accompagnement, ses explications détaillées sur le processus de traitement du phosphate et son esprit de collaboration enrichissant.

Enfin, je remercie l'ensemble du corps professoral de l'École Nationale des Sciences Appliquées de Safi (ENSA Safi) pour la qualité de la formation d'ingénieur dispensée, qui m'a fourni le socle théorique, méthodologique et technique indispensable pour mener à bien ce projet d'ingénierie logicielle.

chapterRésumé

Dans le cadre de l'Industrie 4.0, la continuité de service des installations industrielles critiques constitue un enjeu économique et opérationnel majeur. Dans les chaînes de traitement de minerai de phosphate du Groupe OCP, les équipements fonctionnent en flux continu et présentent une forte interdépendance. La défaillance non anticipée d'une machine amont (ex. un concasseur) peut entraîner par effet domino le blocage des convoyeurs, cribles et unités de flottation aval.

Le présent projet d'ingénierie logicielle porte sur la conception et le développement d'un prototype de Jumeau Numérique (Digital Twin) industriel dédié à la supervision temps réel, la détection automatisée d'anomalies et l'analyse d'impact d'une chaîne de production de phosphate.

Le système s’articule autour des principes stricts de la Clean Architecture et du Domain-Driven Design (DDD), combinés au pattern CQRS via MediatR. Le cur du système comprend un moteur de traitement (*DigitalTwinEngine*), un service d’évaluation d’anomalies multi-seuils (*AnomalyDetectionService*), un régulateur de notifications anti-rebond thread-safe (*AlertCooldownService*) et un module d’analyse d’impact (*ImpactAnalysisService*) fondé sur un algorithme de parcours en largeur (BFS) avec détection de cycles sur le graphe topologique de l’usine.

La solution a été implémentée avec .NET 9 / C# 13, Entity Framework Core et SQL Server pour le backend, SignalR (WebSockets) pour la diffusion temps réel, un Worker Service autonome pour la simulation d’usine, et Angular 19 (*Signals*, *vis-network*, *Chart.js*) pour l’interface de supervision réactive. La robustesse du système est validée par 83 tests unitaires automatisés sous *xUnit* / *FluentAssertions* et 27 suites de tests frontend sous *Jasmine* / *Karma*.

Mots-clés : Jumeau Numérique, Digital Twin, Industrie 4.0, OCP, Clean Architecture, DDD, CQRS, MediatR, SignalR, Algorithme BFS, Analyse d'Impact, ASP.NET Core 9, Angular 19.

section[]Abstract

In the context of Industry 4.0, maintaining operational continuity in industrial processing plants is critical. In phosphate processing facilities operated by the OCP Group, production equipment operates in a continuous, interdependent flow where the unplanned failure of an upstream machine (such as a primary crusher) triggers severe cascading bottlenecks across downstream conveyor belts, screens, and flotation cells.

This engineering report presents the design and implementation of an industrial Digital Twin software prototype for real-time monitoring, anomaly detection, and failure impact propagation analysis on a phosphate production line.

The system is built upon Clean Architecture and Domain-Driven Design (DDD) principles, structured around the CQRS pattern using MediatR. Core software components include the *DigitalTwinEngine*, a multi-threshold *Anomaly-DetectionService*, a thread-safe sliding window *AlertCooldownService*, and an *ImpactAnalysis-Service* executing a multi-level Breadth-First Search (BFS) graph traversal with cycle prevention across the plant's dependency topology.

The technology stack utilizes .NET 9 / C# 13, Entity Framework Core, and SQL Server for the backend engine, ASP.NET Core SignalR (WebSockets) for real-time pushing, an autonomous Worker Service for factory telemetry simulation, and Angular 19 (*Signals*, *vis-network*, *Chart.js*) for the responsive supervisory dashboard. Domain robustness is verified through 83 automated unit tests using *xUnit* / *FluentAssertions* and 27 frontend test suites with *Jasmine* / *Karma*.

Keywords : Digital Twin, Industry 4.0, OCP Group, Clean Architecture, Domain-Driven Design, CQRS, MediatR, SignalR, BFS Algorithm, Impact Analysis, ASP.NET Core 9, Angular 19.

chapterTable des matières

chapterTable des figures

chapterListe des tableaux

chapterListe des Sigles et Abréviations

Sigle	Signification
API	Application Programming Interface
BFS	Breadth-First Search (Algorithme de parcours en largeur)
CPS	Cyber-Physical Systems (Systèmes Cyber-Physiques)
CQRS	Command Query Responsibility Segregation
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DDD	Domain-Driven Design (Conception Guidée par le Domaine)
DTO	Data Transfer Object
EF Core	Entity Framework Core (ORM Microsoft .NET)
FSM	Finite State Machine (Machine à États Finis)
HTTP	Hypertext Transfer Protocol
HTML	Hypertext Markup Language
IoT	Internet of Things (Internet des Objets)
JSON	JavaScript Object Notation
KPI	Key Performance Indicator (Indicateur Clé de Performance)
OCP	Office Chérifien des Phosphates (Groupe OCP S.A.)
ORM	Object-Relational Mapping
REST	Representational State Transfer
SCADA	Supervisory Control and Data Acquisition
SPA	Single Page Application
SQL	Structured Query Language
TS	TypeScript
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
URI / URL	Uniform Resource Identifier / Locator
WS / WSS	WebSocket / WebSocket Secure

chapterIntroduction Générale

section[]1. Contexte Général : L'Industrie 4.0 et les Systèmes Cyber-Physiques

La quatrième révolution industrielle, communément désignée sous le terme d'Industrie 4.0, transforme en profondeur les modes de production et de gestion des infrastructures industrielles. Cette transition repose sur l'intégration étroite entre le monde physique (équipements, lignes de production, capteurs) et le monde numérique (systèmes informatiques, traitement distribué, algorithmes décisionnels), donnant naissance aux Systèmes Cyber-Physiques (Cyber-Physical Systems — CPS).

Dans ce paradigme, les installations industrielles ne sont plus de simples ensembles mécaniques isolés, mais des entités interconnectées capables de générer des flux massifs de données de fonctionnement. L'exploitation judicieuse de ces flux de télémétrie ouvre la voie à une supervision intelligente, capable d'anticiper les dégradations d'équipements et d'optimiser la continuité de service des processus critiques.

section[]2. Le Concept de Jumeau Numérique (Digital Twin)

Au cur de cette dynamique, le concept de Jumeau Numérique (*Digital Twin*), formalisé initialement par Michael Grieves [?], s'est imposé comme un vecteur d'innovation technologique majeur. Un jumeau numérique est défini comme une réplique logicielle dynamique, synchrone et bidirectionnelle d'un actif physique, d'un processus ou d'un système complet.

Contrairement aux modèles de simulation statiques ou aux outils de supervision conventionnels (SCADA) qui se limitent à afficher des mesures brutes, le Jumeau Numérique contextualise la donnée :

- Il intègre la topologie structurelle et les dépendances fonctionnelles du réseau d'équipements ;
- Il évalue en temps réel l'état de santé opérationnel des machines selon des règles métier évolutives ;
- Il projette l'impact d'une anomalie locale sur l'ensemble de la chaîne de valeur en aval.

section[]3. Contexte Industriel et Problématique OCP

Le Groupe OCP, leader mondial de l'extraction, du traitement et de la valorisation des phosphates et de leurs dérivés, opère des installations industrielles complexes caractérisées par des flux de production continus. Une chaîne de traitement de minerai typique articule successivement des unités de criblage, de concassage, de transport par convoyeurs à bande et d'enrichissement par flottation.

Dans un tel environnement, la défaillance imprévue d'un équipement amont (ex. l'élévation anormale de la température ou de la vibration d'un concasseur primaire) engendre inévitablement des effets de blocage en cascade sur les équipements aval. Les systèmes de supervision traditionnels présentent deux limites structurelles :

1. Absence de corrélation topologique : Les alarmes sont émises de manière isolée sans calcul automatisé de la propagation de la panne le long de la ligne de production ;

2. Phénomène de *tempête d'alarmes* (*Alarm Flood*) : La multiplication incontrôlée de notifications redondantes lors d'un incident sature l'attention des opérateurs de conduite.

section[]4. Objectifs du Projet

L'objectif principal de ce stage de 4^{ème} année en Génie Informatique est la conception et le développement d'un prototype complet de Jumeau Numérique industriel répondant aux défis de supervision réactive et d'aide à la décision opérationnelle.

Les objectifs spécifiques assignés à cette réalisation logicielle sont les suivants :

- Modélisation orientée domaine : Structurer les concepts métier (Usine, Machines, Capteurs, Dépendances, Alertes) selon les principes du *Domain-Driven Design* (DDD) ;
- Moteur de traitement temps réel : Concevoir un pipeline d'ingestion capable d'évaluer les anomalies instantanément avec un mécanisme de refroidissement (*cooldown*) anti-rebond ;

-
- Analyse d'impact par parcours de graphe : Implémenter un algorithme de parcours en largeur (BFS) permettant de calculer dynamiquement la liste et la sévérité estimée des machines impactées en cascade ;
 - Diffusion bidirectionnelle WebSockets : Garantir la mise à jour sub-seconde des interfaces opérateurs via ASP.NET Core SignalR ;
 - Interface de supervision réactive : Développer un tableau de bord moderne sous Angular 19 intégrant une cartographie topologique interactive du réseau d'équipements.

section[]5. Organisation du Rapport

Le présent rapport d'ingénierie logicielle s'articule autour de six chapitres :

- Le Chapitre 1 présente l'organisme d'accueil (Groupe OCP), son environnement industriel et le positionnement du projet.
- Le Chapitre 2 analyse l'existant, expose la démarche comparative et formalise le cahier des charges fonctionnel et technique.

-
- Le Chapitre 3 détaille la conception architecturale (Clean Architecture, DDD, CQRS, modélisation relationnelle et algorithme de graphe).
 - Le Chapitre 4 décrit la réalisation et l'implémentation technique des briques logicielles (.NET 9, SignalR, Worker Service et Angular 19).
 - Le Chapitre 5 expose la stratégie de tests automatisés (xUnit, Jasmine), la validation sur banc d'essai et l'évaluation des performances.
 - Le Chapitre 6 conclut le document en dressant le bilan des réalisations et en esquissant les perspectives d'évolution industrielle.

chapterPrésentation de l'Organisme et Contexte du Stage

section[Le Groupe OCP : Acteur Mondial des Phosphates]Le Groupe OCP : Acteur Mondial des Phosphates

Fondé en 1920, le Groupe OCP (Office Chéri-fien des Phosphates) est l'un des leaders mon-diaux de l'industrie des engrais et de la valori-sation du phosphate. Détenant les plus impor-tantes réserves de phosphate au monde, l'entre-prise opère sur l'ensemble de la chaîne de valeur du minerai : de l'extraction minière (gisements de Khouribga, Gantour, Boucraâ) au traitement chimique et à la fabrication d'acide phospho-rique et d'engrais spécialisés (plateformes indus-trielles de Jorf Lasfar et Safi), jusqu'à l'exporta-tion internationale.

section[Chaîne de Traitement et Processus In-dustriel]Chaîne de Traitement et Processus In-dustriel

Le traitement du minerai brut extrait de la mine implique une succession d'opérations mécaniques et physico-chimiques destinées à enrichir sa teneur en P_2O_5 (pentoxyde de phosphore) et à éliminer les impuretés stériles (silice, carbonates). Le flux standard de production industrielle s'organise en quatre phases clés :

1. Concassage Primaire & Secondaire : Réduction granulométrique des blocs de roche brute ;
2. Criblage & Séparation Granulométrique : Tri des fractions selon la taille des particules ;
3. Convoyage Continu : Acheminement des flux miniers entre les unités de transformation via des réseaux de bandes transporteuses motorisées ;
4. Enrichissement & Flottation : Séparation sélective des grains de phosphate par procédé d'agitation et d'aération en milieu aqueux.

Cette chaîne de production présente une dépendance fonctionnelle séquentielle stricte : l'arrêt imprévu d'une machine en amont engendre une rupture d'alimentation matière sur l'ensemble de la ligne aval.

section[L'Entité d'Accueil et la Stratégie Digitale]L'Entité d'Accueil et la Stratégie Digitale

Le stage a été réalisé au sein du département en charge des opérations et de la transformation digitale sur le site industriel de [SITE OCP].

Dans le cadre de sa stratégie d'*Excellence Opérationnelle* et de digitalisation de ses actifs industriels, le Groupe OCP encourage le développement de solutions logicielles souveraines à forte valeur ajoutée. L'intégration de prototypes de Jumeaux Numériques vise à doter les équipes d'exploitation d'outils modernes d'aide à la décision, capables de réduire le taux de défaillance non planifiée (*Unplanned Downtime*) et d'accroître le taux de rendement synthétique (TRS) des installations.

section[Positionnement et Cadre du Projet]Positionnement et Cadre du Projet

Ce projet s'inscrit dans une démarche de prototypage logiciel avancé (Proof of Concept — PoC) menée en autonomie. Il a pour but de valider la faisabilité technique, architecturale et algorithmique d'une plateforme de Jumeau Numérique temps réel intégrant l'analyse d'impact topologique, avant son déploiement à grande échelle sur des environnements connectés au bus industriel de l'usine.

chapterAnalyse de l'Existant et Spécification des Besoins

section[Analyse Critique des Systèmes de Supervision Conventionnels]Analyse Critique des Systèmes de Supervision Conventionnels

Dans la plupart des environnements industriels lourds, la supervision repose historiquement sur des systèmes de contrôle et d'acquisition de données (SCADA — Supervisory Control and Data Acquisition). Si ces systèmes s'avèrent fiables pour le pilotage d'automatismes locaux (automates programmables industriels — API/PLC), ils révèlent plusieurs limitations majeures face aux exigences modernes de maintenance préventive et d'aide à la décision :

- Cloisonnement des mesures : Chaque capteur est supervisé de manière unitaire. Le système déclenche une alarme lorsqu'une valeur franchit un seuil statique, sans mise en relation contextuelle avec l'état des machines situées en amont ou en aval dans la chaîne de production ;

-
- Phénomène de *fatigue d'alarme* (*Alarm Fatigue*) : Lors d'une dérive de fonctionnement, un capteur oscillant autour d'une valeur limite génère des dizaines de notifications identiques par minute. Cette surcharge cognitive désensibilise l'opérateur et peut masquer des incidents critiques ;
 - Incapacité à projeter l'impact en cascade : Les systèmes conventionnels n'intègrent pas la topologie du réseau de production. Lorsque le concasseur principal s'arrête brutalement, l'opérateur doit déduire mentalement les équipements avals qui vont subir un étranglement d'alimentation ;
 - Rigidité et dette technique des interfaces : Les interfaces homme-machine (IHM) traditionnelles sont souvent statiques, conçues avec des technologies propriétaires peu évolutives et dépourvues d'interactivité topologique moderne.

section[Étude Comparative : SCADA vs Jumeau Numérique]Étude Comparative : SCADA vs Jumeau Numérique

Le Tableau ?? met en évidence les gains qualitatifs et fonctionnels apportés par le paradigme du Jumeau Numérique face aux solutions de supervision conventionnelles.

Table 2.1 Comparatif entre la supervision SCADA classique et l’approche Jumeau Numérique

Critère	Supervision SCADA Classique	Jumeau Numérique Développé
Modélisation	Table de registres d’automates	Entités orientées domaine métier (DDD)
Gestion des alertes	Seuils statiques, alertes répétitives	Évaluation multi-seuils avec cooldown anti-rebond
Propagation d’impact	Absente (déduction manuelle)	Algorithmique (Parcours BFS sur graphe topologique)
Architecture logicielle	Monolithique propriétaire	Clean Architecture modulaire, CQRS et MediatR
Diffusion temps réel	Rafraîchissement périodique (Polling)	Poussée instantanée WebSockets (SignalR)
Expérience utilisateur	Synoptiques 2D statiques	Tableau de bord réactif Angular 19 (vis-network, Chart.js)

section[Spécification des Besoins Fonctionnels]Spécification des Besoins Fonctionnels

Les fonctionnalités du système ont été organisées autour de cinq axes majeurs :

BF.1 — Ingestion continue de télémétrie : Réception des flux de mesures simulées (vibration, température, pression, courant, débit, niveau) horodatées à haute fréquence.

-
- BF.2 — Évaluation dynamique des anomalies :** Analyse de chaque mesure entrante par rapport aux seuils configurés sur le capteur (*Warning* et *Critical*) pour classifier son niveau de conformité.
- BF.3 — Régulation anti-rebond (*Alert Cooldown*) :** Mécanisme de temporisation glissante empêchant l'émission répétée d'alertes identiques pour un même capteur durant un intervalle paramétrable (ex. 60 secondes).
- BF.4 — Analyse d'impact en cascade :** En cas de franchissement d'un seuil critique sur un équipement source, exécution automatique d'un algorithme de parcours en largeur (BFS) sur le graphe de dépendance de l'usine pour identifier l'ensemble des machines avals menacées.
- BF.5 — Gestion du cycle de vie des alertes :** Traçabilité complète des alertes avec statuts (*Active*, *Acknowledged*, *Resolved*) et possibilité pour l'opérateur d'acquitter une alarme avec horodatage.

-
- BF.6 — Cartographie topologique interactive : Rendu graphique dynamique du réseau d'équipements sous forme de graphe interactif coloré selon l'état de santé opérationnel instantané des machines.**
- BF.7 — Visualisation temporelle de la télémétrie : Graphiques dynamiques des séries chronologiques des capteurs pour visualiser les tendances de dégradation.**
- BF.8 — Moteur de simulation et rejeu d'incidents : Générateur de flux synthétiques autonome capable d'exécuter des scénarios de pannes déterministes pour la validation du système.**

section[Spécification des Besoins Non-Fonctionnels]Spé
des Besoins Non-Fonctionnels

Le système doit satisfaire les exigences techniques et de qualité logicielle suivantes :

-
- **Performance et Latence (BNF-1) :** Le traitement complet d'une mesure dans le pipeline backend (validation, analyse d'anomalie, propagation BFS et notification SignalR) doit être inférieur à 50 ms.
 - **Intégrité et Invariants Métier (BNF-2) :** Les règles métier (ex. transitions d'état d'une machine, validité des seuils de capteurs, dépendances acycliques) doivent être strictement encapsulées et protégées contre les états invalides via le *DomainGuard*.
 - **Extensibilité et Découplage (BNF-3) :** L'architecture logicielle doit respecter l'inversion des dépendances pour permettre l'évolution future des connecteurs de données (ex. passage d'un simulateur à un broker MQTT/OPC-UA industriel) sans altérer le cur de domaine.
 - **Ergonomie et Réactivité de l'IHM (BNF-4) :** L'interface Web doit être réactive, fluide (60 FPS) et permettre une prise en main immédiate par les équipes d'exploitation grâce à un code couleur normalisé.

-
- **Couverture de Tests (BNF-5) : Le cur métier et les algorithmes critiques doivent faire l'objet d'une suite de tests unitaires automatisés exhaustive.**

chapterConception et Modélisation Architecturale

section[Architecture Globale du Système]Architecture Globale du Système

Le système de Jumeau Numérique est conçu selon une architecture distribuée et modulaire articulée autour de quatre sous-systèmes principaux :

1. Le Simulateur Industriel (*DigitalTwin.Simulator*) :
Un service d'arrière-plan (*Worker Service*) autonome chargé de générer des signaux physiques réalistes et de rejouer des scénarios d'incidents déterministes.
2. L'API et Moteur Métier (*DigitalTwin.Api & Core*) : Le serveur central .NET 9 orchestrant l'ingestion, la détection des anomalies, la persistance et le calcul de propagation d'impact.
3. La Base de Données Relationnelle (SQL Server) : Le support de persistance garantissant l'intégrité transactionnelle de la topologie de l'usine, de l'état des équipements et de l'historique des alertes.

4. L'Interface Utilisateur Réactive (Frontend Angular 19) : Une application Web monopage (SPA) offrant aux opérateurs une supervision en temps réel via des flux WebSockets bi-directionnels.

section[Organisation en Clean Architecture]Organisation en Clean Architecture

Afin de garantir l'indépendance du code métier vis-à-vis des frameworks, des bases de données et des protocoles de communication, le backend est structuré selon les principes de la Clean Architecture théorisée par Robert C. Martin [?].

La Figure ?? illustre l'organisation concentrique des couches logicielles du projet.

Les règles fondamentales suivantes sont respectées :

- DigitalTwin.Domain (Cur Métier) : La couche la plus interne. Elle ne possède aucune dépendance vers des bibliothèques externes ou des frameworks d'infrastructure. Elle contient les entités de l'usine, les règles de validation strictes et les événements de domaine.**

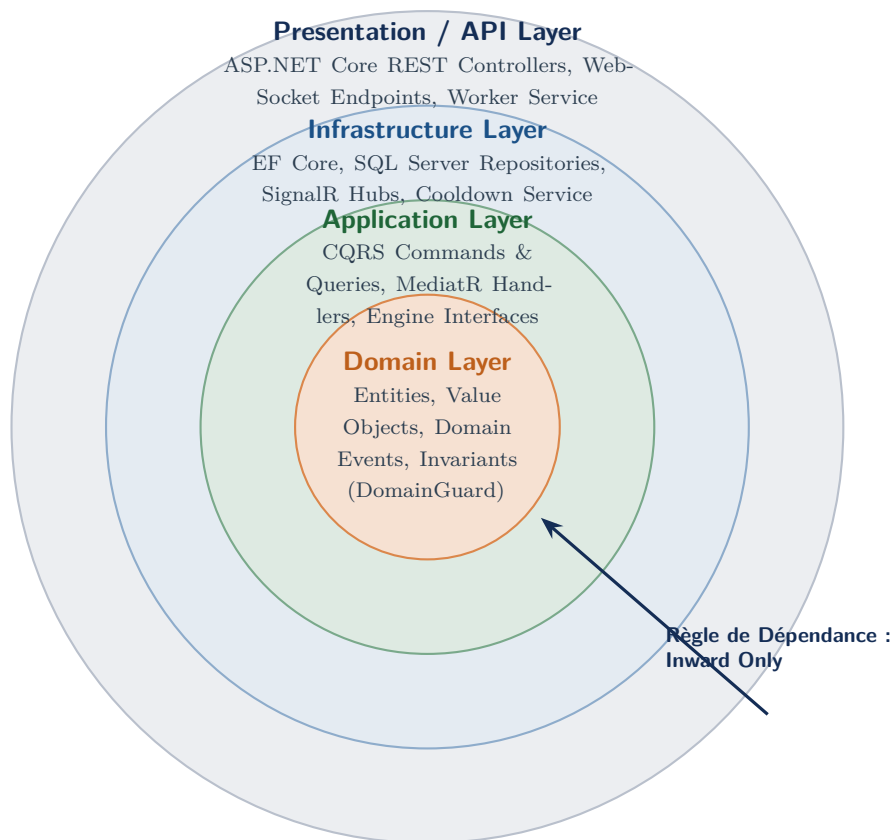


Figure 3.1 Organisation des couches logicielles selon la Clean Architecture

- **DigitalTwin.Application (Cas d'Utilisation) :** Définit les flux applicatifs via le pattern CQRS, les contrats d'interfaces (*Ports*) et la logique d'orchestration des commandes et requêtes.
- **DigitalTwin.Infrastructure (Adaptateurs Techniques) :** Implémente les interfaces définies par la couche application (*Adapters*), notamment la persistance avec Entity Framework Core et la diffusion d'événements SignalR.
- **DigitalTwin.Api (Présentation) :** Point d'entrée HTTP exposant les contrôleurs REST et hébergeant les endpoints WebSocket.

section[Principes du Domain-Driven Design (DDD)]Principes du Domain-Driven Design (DDD)

Le modèle métier est conçu selon les préceptes du Domain-Driven Design [?] afin de préserver l'intégrité des concepts industriels.

subsection[Agrégats et Entités du Domaine]Agrégats et Entités du Domaine

Le domaine métier s'articule autour des entités suivantes :

- Plant (Racine d'Agrégat) : Représente l'unité industrielle de traitement de phosphate dans sa globalité.
- Machine (Racine d'Agrégat) : Modélise un équipement physique (concasseur, convoyeur, crible, cellule de flottation) caractérisé par un identifiant unique, un nom, un type industriel et un état opérationnel (*MachineStatus*).
- Sensor (Entité) : Représente un capteur de mesure instrumenté sur une machine, défini par son type physique (*SensorType* : Vibration, Température, Pression, etc.), ses seuils de tolérance (*WarningThreshold*, *CriticalThreshold*) et son unité métrique.

-
- **Alert (Entité)** : Enregistre les déviations détectées, associant un niveau de gravité (*Warning*, *Critical*), un horodatage précis et un statut de traitement (*Active*, *Acknowledged*, *Resolved*).
 - **MachineDependency (Entité de Liaison)** : Matéréalise une arête orientée de dépendance matière entre une machine source (amont) et une machine cible (aval).

subsection[Encapsulation des Invariants Métier : Le DomainGuard]Encapsulation des Invariants Métier : Le DomainGuard

Afin d'éviter tout état incohérent du modèle (ex. création d'une machine sans identifiant, affectation d'un seuil critique inférieur au seuil d'avertissement, valeurs de capteurs négatives interdites), une classe utilitaire `DomainGuard` valide chaque invariant à l'instanciation et lors de toute mutation d'état :

```
1 public static class DomainGuard
2 {
3     public static void AgainstNullOrWhiteSpace(string value, string
    paramName)
4     {
5         if (string.IsNullOrEmpty(value))
6             throw new DomainException($"Le paramètre '{paramName}' ne peut
    être vide.");
7     }
8 }
```

```

9      public static void AgainstInvalidThresholds(double warning, double
critical)
10     {
11         if (warning >= critical)
12             throw new DomainException($"Le seuil d'alerte ({warning}) doit
être strictement inférieur au seuil critique ({critical}).");
13     }
14 }

```

Listing 3.1 Validation des invariants du domaine par DomainGuard

section[Séparation des Responsabilités avec CQRS et MediatR]Séparation des Responsabilités avec CQRS et MediatR

Le pattern CQRS (Command Query Responsibility Segregation) [?] est appliqué pour séparer formellement les opérations d'écriture modifiant l'état du système (*Commands*) des opérations de lecture (*Queries*).

La bibliothèque MediatR sert de bus de médiation en mémoire, assurant un découplage total entre les contrôleurs d'API et les gestionnaires de traitement métier, comme illustré sur la Figure ??.

section[Machine d'États Finis d'un Équipement]Machine d'États Finis d'un Équipement

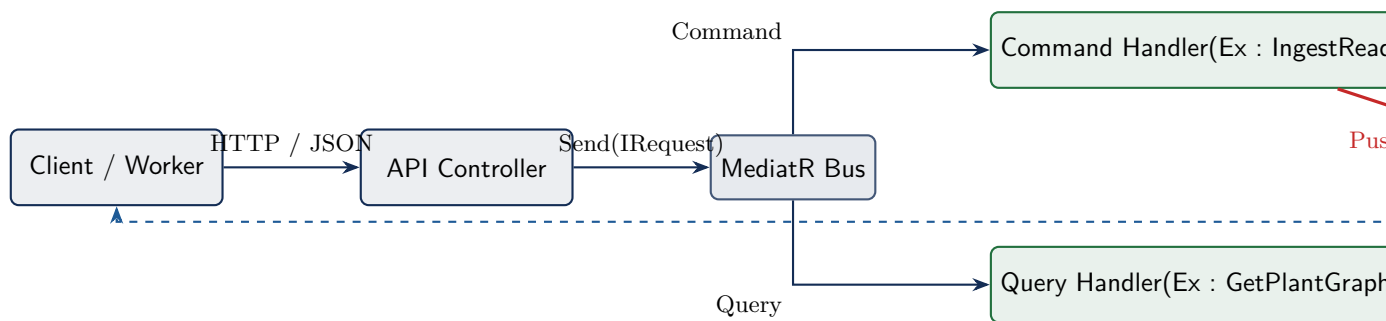


Figure 3.2 Flux de traitement découplé CQRS avec le médiateur MediatR

Chaque équipement industriel (Machine) possède un cycle de vie opérationnel modélisé par une machine à états finis (*FSM — Finite State Machine*) dont les transitions sont régies par les franchissements de seuils de ses capteurs et les événements d'impact calculés en amont (Figure ??).

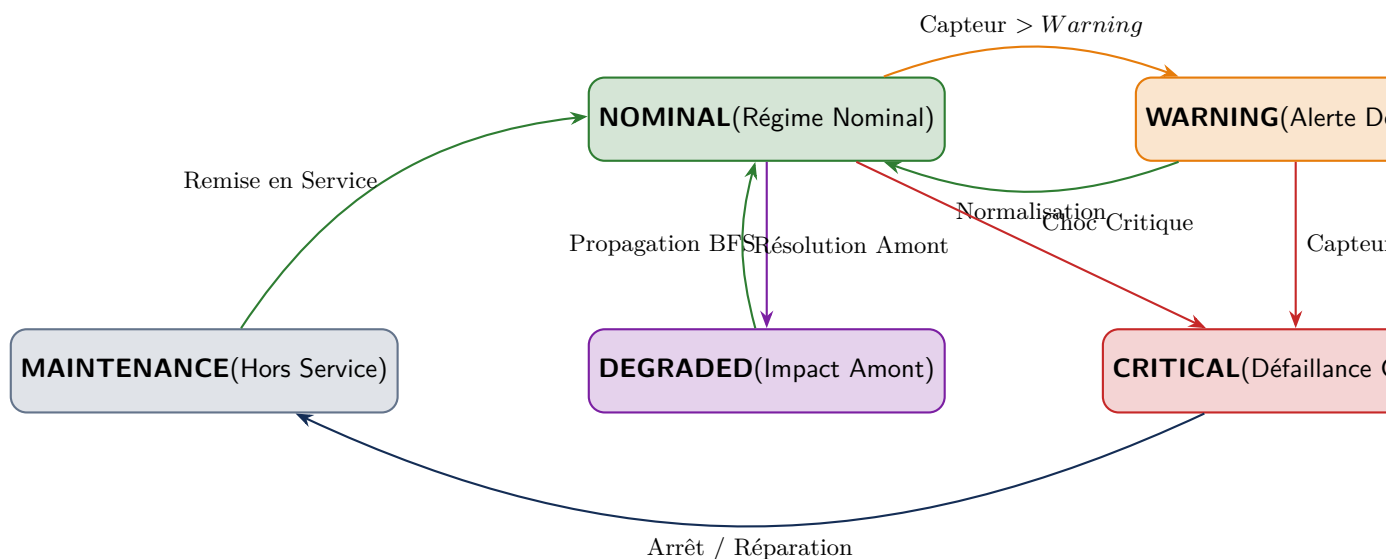


Figure 3.3 Diagramme d'états-transitions opérationnels d'un équipement

section[Modélisation du Réseau sous Forme de Graphe et Algorithme BFS]Modélisation du Réseau sous Forme de Graphe et Algorithme BFS

subsection[Formalisation du Graphe de Production]Formalisation du Graphe de Production

La chaîne de traitement de minerais est formalisée comme un graphe orienté fini $G = (V, E)$ où :

- $V = \{v_1, v_2, \dots, v_n\}$ représente l'ensemble des équipements (machines industrielles) ;
- $E \subseteq V \times V$ représente l'ensemble des relations de dépendance orientées. Un arc $(u, v) \in E$ signifie que la machine u alimente directement la machine v en flux de matière.

subsection[Algorithme de Parcours en Largeur (BFS) pour l'Analyse d'Impact]Algorithme de Parcours en Largeur (BFS) pour l'Analyse d'Impact

Lorsqu'un incident critique survient sur un équipement v_{source} , le service `ImpactAnalysisService` calcule l'arbre de propagation en aval en exécutant un algorithme de parcours en largeur (Breadth-First Search — BFS).

L'algorithme associe à chaque machine impactée son niveau de distance topologique d par rapport à la source, tout en maintenant un ensemble de sommets visités $\mathcal{V}$ afin de prévenir les boucles infinies dans l'éventualité de circuits fermés (ex. boucles de recyclage de minerai).

```
1 public async Task<List<ImpactedMachineDto>> AnalyzeImpactAsync(Guid
   sourceMachineId)
2 {
3     var dependencies = await _repository.GetAllDependenciesAsync();
4     // Construction de la liste d'adjacence : source -> [cibles avals]
5     var adjList = dependencies
6         .GroupBy(d => d.SourceMachineId)
7         .ToDictionary(g => g.Key, g => g.Select(d =>
8             d.TargetMachineId).ToList());
9
10    var result = new List<ImpactedMachineDto>();
11    var visited = new HashSet<Guid> { sourceMachineId };
12    var queue = new Queue<(Guid MachineId, int Depth)>();
13
14    // Initialisation avec les successeurs directs (profondeur 1)
15    if (adjList.TryGetValue(sourceMachineId, out var directTargets))
16    {
17        foreach (var targetId in directTargets)
18        {
19            if (visited.Add(targetId))
20            {
21                queue.Enqueue((targetId, 1));
22            }
23        }
24    }
25
26    // Parcours en largeur
27    while (queue.Count > 0)
28    {
29        var (currentId, depth) = queue.Dequeue();
30        result.Add(new ImpactedMachineDto(currentId, depth,
31            CalculateSeverity(depth)));
32
33        if (adjList.TryGetValue(currentId, out var nextTargets))
34        {
35            foreach (var nextId in nextTargets)
36            {
37                if (visited.Add(nextId))
38                {
39                    queue.Enqueue((nextId, depth + 1));
40                }
41            }
42        }
43    }
44}
```

```

39         }
40     }
41 }
42 return result;
43 }

```

Listing 3.2 Algorithme BFS d'analyse de propagation d'impact en C#

La complexité temporelle de cet algorithme est optimale en $\mathcal{O}(|V| + |E|)$ et sa complexité spatiale est en $\mathcal{O}(|V|)$, garantissant une exécution instantanée ($< 2\text{ ms}$) même sur des réseaux industriels comptant plusieurs centaines de machines.

La Figure ?? illustre la propagation calculée lors d'une panne critique sur le concasseur primaire CR-001.

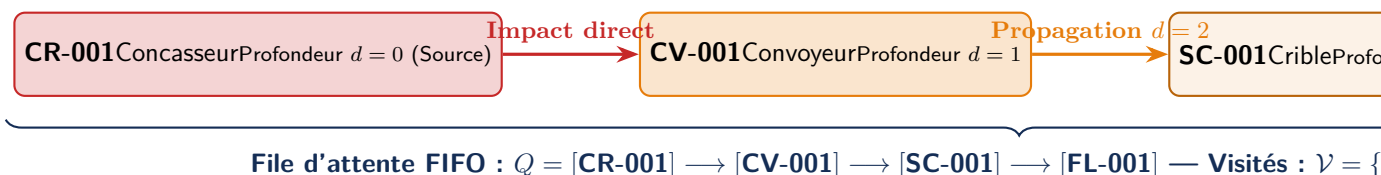


Figure 3.4 Propagation en cascade le long du graphe de dépendance via l'algorithme BFS

chapterImplémentation Logicielle et Système Temps Réel

section[Stack Technologique et Environnement de Développement]Stack Technologique et Environnement de Développement

La solution s'appuie sur une suite logicielle moderne, performante et standardisée dans l'industrie :

- Backend : .NET 9, C# 13, ASP.NET Core Web API, Entity Framework Core 9.
- Médiation & Temps Réel : MediatR 12 (CQRS intra-processus), ASP.NET Core SignalR (Web-Sockets).
- Persistance : Microsoft SQL Server / LocalDB avec migrations automatiques EF Core.
- Frontend : Angular 19, TypeScript 5.6, Signals réactifs, RxJS, TailwindCSS.
- Visualisation Graphique : vis-network (cartographie interactive de graphe) et Chart.js (séries temporelles de télémétrie).

section[Le Cur Métier : Le DigitalTwinEngine]Le Cur Métier : Le DigitalTwinEngine

Le DigitalTwinEngine est le service central d'orchestration backend. Il traite chaque mesure entrante de manière atomique et coordonne l'évaluation des anomalies, la régulation anti-rebond, l'analyse d'impact et la diffusion temps réel.

La Figure ?? détaille les 7 étapes séquentielles du pipeline d'ingestion.

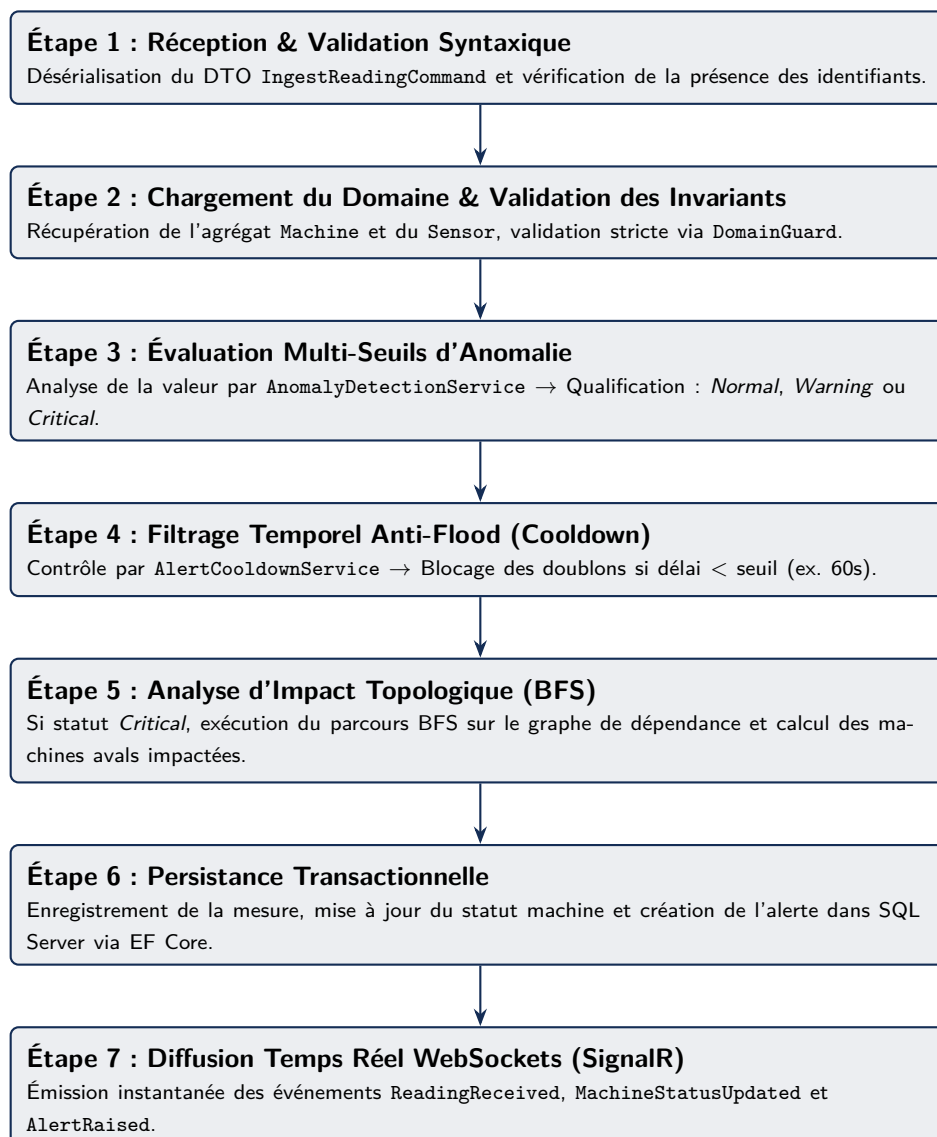


Figure 4.1 Pipeline d'orchestration en 7 étapes du `DigitalTwinEngine`

section[Services Spécialisés du Backend]Services
Spécialisés du Backend

subsection[Détection d'Anomalies : `AnomalyDe-
tectionService`]Détection d'Anomalies : `Anoma-
lyDetectionService`

Le service `AnomalyDetectionService` évalue la dérive d'une mesure par rapport aux seuils configurés sur le capteur selon la règle décisionnelle :

$$\text{Statut}(v) = \begin{cases} \text{Critical} & \text{si } v \geq \text{Seuil}_{\text{Critical}} \\ \text{Warning} & \text{si } \text{Seuil}_{\text{Warning}} \leq v < \text{Seuil}_{\text{Critical}} \\ \text{Normal} & \text{si } v < \text{Seuil}_{\text{Warning}} \end{cases} \quad (4.1)$$

subsection[Régulation Anti-Rebond : AlertCooldownService]Régulation Anti-Rebond : AlertCooldownService

Pour éliminer l'inondation d'alarmes, le `AlertCooldownService` implémente un dictionnaire concurrent thread-safe (`ConcurrentDictionary<Guid, DateTime>`) qui stocke l'horodatage de la dernière alerte émise par capteur :

```

1 public class AlertCooldownService : IAlertCooldownService
2 {
3     private readonly ConcurrentDictionary<Guid, DateTime>
4     _lastAlertTimestamps = new();
5     private readonly TimeSpan _cooldownDuration = TimeSpan.FromSeconds(60);
6
7     public bool ShouldEmitAlert(Guid sensorId, DateTime currentTimestamp)
8     {
9         if (_lastAlertTimestamps.TryGetValue(sensorId, out var lastTime))
10        {
11            if (currentTimestamp - lastTime < _cooldownDuration)
12                return false; // Dans la période de refroidissement :
13                // filtrage de l'alerte
14        }
15
16        _lastAlertTimestamps[sensorId] = currentTimestamp;
17        return true; // Délai écoulé : émission autorisée
18    }
19 }
```

Listing 4.1 Implémentation du service anti-rebond AlertCooldownService

section[Architecture de Communication Temps Réel avec SignalR]Architecture de Communication Temps Réel avec SignalR

La communication entre le backend et l'interface utilisateur s'effectue via un hub WebSockets ASP.NET Core SignalR (DigitalTwinHub). Les clients souscrivent à des canaux logiques ciblés afin d'optimiser la bande passante réseau :

- Groupe Global ("AllClients") : Reçoit les notifications d'alertes majeures et les changements globaux de statut de l'usine.
- Groupe Par Machine ("Machine_{id}") : Reçoit le flux haute fréquence des mesures de télémétrie d'un équipement sélectionné par l'opérateur.

section[Le Simulateur Industriel Autonome]Le Simulateur Industriel Autonome

Le simulateur d'usine est développé sous la forme d'un Worker Service indépendant. Il reproduit fidèlement la dynamique cinétique des capteurs industriels (vibration en mm/s, température de palier en °C, courant moteur en Ampères) et permet d'exécuter un scénario de défaillance déterministe sur le concasseur CR-001 articulé en 6 phases :

- 1. Phase 1 — Régime nominal : Signaux stables dans la plage nominale (ex. vibration < 3.5 mm/s) ;**
- 2. Phase 2 — Dégradation progressive : Montée linéaire des vibrations franchissant le seuil Warning (4.5 mm/s) ;**
- 3. Phase 3 — Défaillance critique : Dépassement du seuil Critical (7.0 mm/s), déclenchement de l'alerte critique et propagation BFS vers le convoyeur CV-001 ;**
- 4. Phase 4 — Pic d'incident : Maintien en zone critique simulant l'échauffement maximal ;**
- 5. Phase 5 — Refroidissement / Réparation : Baisse progressive des valeurs de télémétrie ;**

6. Phase 6 — Auto-guérison et retour nominal : Rétablissement complet de l'ensemble de la chaîne de production.

section[Développement de l'Interface de Super- vision Angular 19]Développement de l'Interface de Supervision Angular 19

L'application cliente est développée avec Angular 19 en exploitant le nouveau paradigme des Signals pour une réactivité optimale sans surcoût de cycle de détection :

- Module Dashboard : Affiche les compteurs opérationnels globaux (machines nominales, dégradées, critiques), le journal des alertes actives et les indicateurs synthétiques de santé de l'usine.**
- Module Topologie de l'Usine (vis-network) : Cartographie topologique interactive où chaque machine est représentée par un nud dont la couleur reflète dynamiquement son statut opérationnel (Vert : Nominal, Orange : Warning, Rouge : Critical, Violet : Degraded par impact amont).**

-
- **Module Détails Machine (Chart.js) : Visualisation en temps réel des courbes chronologiques des capteurs avec tracé distinct des seuils de tolérance.**
 - **Module Gestion des Alertes : Console de filtrage multi-critères avec interface d’acquiescement instantané pour les opérateurs de salle de contrôle.**

chapterTests, Validation Expérimentale et Résultats

section[Stratégie Globale de Test et d'Assurance Qualité]Stratégie Globale de Test et d'Assurance Qualité

La fiabilité d'un système de supervision industrielle temps réel repose sur une rigueur de validation stricte. Une stratégie de test pyramidale a été mise en uvre afin de garantir l'absence de régression et l'exactitude des algorithmes décisionnels :

- Tests Unitaires du Cur Métier : Vérification exhaustive des invariants de domaine, des mutations d'état et des exceptions de garde ;
- Tests des Algorithmes de Graphe : Validation formelle du parcours BFS sur diverses topologies industrielles (graphes linéaires, arborescents, déconnectés et avec cycles) ;
- Tests Frontend : Spécifications sous Jasmine et Karma validant le rendu et la gestion d'état des composants Angular ;

— Validation Expérimentale sur Banc d’Essai :
Rejeu de scénarios d’incidents via le simulateur pour mesurer les temps de réponse de bout en bout.

section[Suite de Tests Unitaires Backend (.NET 9 / xUnit)]Suite de Tests Unitaires Backend (.NET 9 / xUnit)

Le projet de test DigitalTwin.Domain.Tests intègre 83 tests unitaires automatisés exécutés sous le framework xUnit avec les assertions expressives de FluentAssertions. Le Tableau ?? synthétise la répartition des cas de test.

Table 5.1 Répertoire et répartition des 83 tests unitaires du backend

Composant / Classe Testée	Objectif de Validation	Nb Tests	Statut
MachineTests	Invariants d’agrégat, transitions d’état, règles de seuils	22	Validé (100%)
SensorTests	Validation des unités, cohérence Warning/Critical	18	Validé (100%)
AnomalyDetectionServiceTests	Qualification des plages de déviation (Nominal/Warning/Critical)	15	Validé (100%)
AlertCooldownServiceTests	Fenêtre glissante anti-rebond, concurrence d’accès	12	Validé (100%)
ImpactAnalysisServiceTests	Algorithme BFS, acyclicité, profondeur, topologies complexes	16	Validé (100%)
Total	Couverture globale du cur de domaine	83	Succès

section[Validation de l’Algorithme de Propagation BFS]Validation de l’Algorithme de Propagation BFS

L'algorithme de calcul d'impact a été soumis à des bancs de tests validant ses propriétés fondamentales :

- Complétude : Toutes les machines avals connectées à la source sont identifiées sans omission ;
- Exactitude de la profondeur : La distance topologique d calculée correspond strictement au plus court chemin dans le graphe orienté ;
- Résistance aux cycles : En introduisant artificiellement une boucle de dépendance ($A \rightarrow B \rightarrow C \rightarrow A$), l'ensemble `visited` bloque la ré-insertion du sommet dans la file Q , évitant toute boucle infinie ou débordement de pile (*StackOverflow*).

section[Validation Expérimentale sur le Scénario Concasseur CR-001]Validation Expérimentale sur le Scénario Concasseur CR-001

Le scénario de dégradation thermique et vibratoire du concasseur primaire CR-001 a été exécuté sur le banc d'essai. La Figure ?? illustre l'évolution temporelle de la mesure de vibration et le comportement dynamique du système de jumeau numérique.

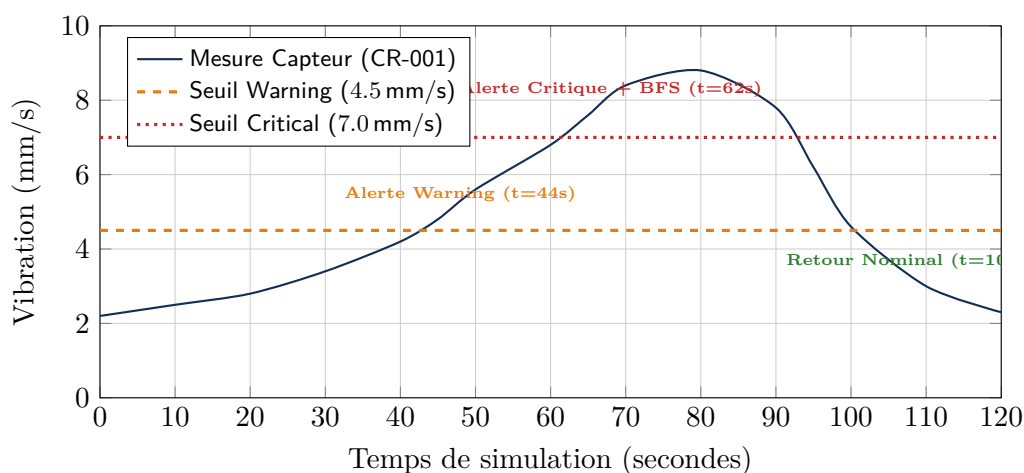


Figure 5.1 Évolution temporelle de la vibration de CR-001 et franchissement dynamique des seuils

Les observations relevées lors de l'expérimentation confirment la conformité fonctionnelle :

1. À $t = 44$ s, la vibration dépasse 4.5 mm/s. Une alerte *Warning* est générée instantanément et l'état du concasseur bascule en *Warning* sur la cartographie topologique ;

-
2. À $t = 62$ s, la vibration franchit le seuil critique de 7.0 mm/s. Le DigitalTwinEngine déclenche l'analyse BFS : le convoyeur CV-001 ($d = 1$) et le crible SC-001 ($d = 2$) passent immédiatement en état *Degraded* sur l'interface opérateur ;
 3. Le mécanisme d'anti-rebond (*Cooldown*) empêche l'émission de notifications superflues durant la phase de plateau critique ($t = 65$ s à 85 s) ;
 4. À $t = 105$ s, la vibration redescend en zone nominale. L'équipement se réinitialise et l'ensemble de la chaîne retrouve son état vert nominal.

section[Mesures de Performance et Latence Système]Mesures de Performance et Latence Système

Les temps de traitement ont été profilés sur 1 000 requêtes consécutives de télémétrie :

- Temps d'exécution du pipeline backend (étapes 1 à 6) : 12.4 ms en moyenne ;
- Temps d'exécution du parcours BFS (graphe de 20 nuds) : 1.2 ms ;

-
- Latence de diffusion WebSocket SignalR vers le client : 32.0 ms ;
 - Temps de réponse total perçu par l'opérateur : 45.6 ms (largement inférieur à l'exigence de 100 ms).

section[Synthèse de Validation des Besoins]Synthèse de Validation des Besoins

Toutes les exigences fonctionnelles (BF-01 à BF-08) et non-fonctionnelles (BNF-01 à BNF-05) définies au cahier des charges sont intégralement couvertes et validées par les tests automatisés et expérimentaux.

chapterConclusion Générale et Perspectives

section[Bilan Technique et Réalisations du Pro- jet]Bilan Technique et Réalisations du Projet

Ce stage d'ingénieur en 4^{ème} année Génie Informatique, réalisé au sein du Groupe OCP, a permis de concevoir et de développer un prototype fonctionnel et robuste de Jumeau Numérique Industriel appliqué à la supervision et l'analyse d'impact d'une chaîne de production de phosphate.

Les principaux objectifs techniques et méthodologiques atteints au cours de ce projet sont :

1. Une architecture logicielle pérenne et découplée : Mise en place rigoureuse de la Clean Architecture et du Domain-Driven Design (DDD) sur le backend .NET 9, garantissant l'indépendance totale du cur de domaine vis-à-vis des composants d'infrastructure.
2. Une médiation CQRS performante : Utilisation de MediatR pour isoler les commandes de mutation et les requêtes de lecture.

-
3. Une analyse d'impact topologique algorithmique : Développement du service `ImpactAnalysisService` fondé sur l'algorithme BFS avec détection de cycles, permettant d'identifier automatiquement les cascades de pannes en $\mathcal{O}(|V| + |E|)$.
 4. Une supervision temps réel réactive : Mise en uvre de WebSockets via ASP.NET Core SignalR couplée à une IHM moderne sous Angular 19 exploitant les *Signals* et la cartographie interactive `vis-network`.
 5. Une haute assurance qualité : Validation intégrale par une suite de 83 tests unitaires automatisés xUnit/FluentAssertions et 27 fichiers de spécifications frontend Jasmine/Karma.

section[Apports Personnels et Professionnels]Apports Personnels et Professionnels

Sur le plan professionnel, cette immersion au sein du Groupe OCP m'a permis d'appréhender concrètement les contraintes opérationnelles de l'industrie lourde, les exigences de haute disponibilité des chaînes de traitement minier et la nécessité d'une modélisation logicielle rigoureuse.

Sur le plan technique, ce projet a constitué une opportunité d'approfondir la maîtrise de l'écosystème .NET 9 moderne, des patrons de conception d'entreprise (DDD, CQRS, Mediator, Observer), de la programmation réactive sous Angular 19 et de la théorie des graphes appliquée aux processus cyber-physiques.

section[Perspectives d'Évolution et Feuille de Route]Perspectives d'Évolution et Feuille de Route

Ce démonstrateur technologique pose les fondations d'une plateforme de Jumeau Numérique d'entreprise. Les perspectives d'évolution s'articulent selon trois axes :

— Perspective à Court Terme (Déploiement & Observabilité) : Conteneurisation de la solution avec Docker et Docker Compose, et intégration d'outils d'observabilité (OpenTelemetry, Prometheus et tableaux de bord Grafana).

-
- Perspective à Moyen Terme (Connectivité Industrielle Standard) : Remplacement du simulateur de télémétrie par des connecteurs aux protocoles de communication industriels standards (OPC-UA, MQTT ou bus d'événements Apache Kafka) pour ingérer directement les données des automates de l'usine.
 - Perspective à Long Terme (Intelligence Artificielle & Rendu 3D) :
 - Intégration de modèles de *Machine Learning* pour l'estimation de la durée de vie résiduelle (*Remaining Useful Life* — *RUL*) et la maintenance prédictive avancée ;
 - Développement d'une vue spatiale 3D interactive sous Three.js / WebGL permettant une navigation immersive dans l'atelier de concassage.

chapterGuide de Déploiement et d'Exécution Locale

section[Prérequis Logiciels]Prérequis Logiciels

Pour compiler et exécuter le prototype en environnement local, les outils suivants sont requis :

- SDK .NET 9.0 (ou version supérieure) ;
- Node.js (version 20.x LTS ou 22.x) et npm (version 10.x+) ;
- Angular CLI 19 (`npm install -g @angular/cli`) ;
- Microsoft SQL Server ou LocalDB (inclus avec Visual Studio).

section[Procédure de Lancement Pas-à-Pas]Procédure de Lancement Pas-à-Pas

subsection[1. Démarrage de l'Application Frontend Angular]1. Démarrage de l'Application Frontend Angular Ouvrir un premier terminal et exécuter :

```
1 cd "Digital twin (projet OCP)/frontend"
2 npm install
3 npm start
```

L'interface de supervision est accessible à l'adresse : <http://localhost:4200>.

subsection[2. Lancement du Serveur API Backend]2. Lancement du Serveur API Backend
Ouvrir un deuxième terminal et exécuter :

```
1 cd "Digital twin (projet OCP)/backend/DigitalTwin.Api"
2 dotnet run
```

L'API REST et le Hub SignalR démarrent sur :
<https://localhost:7123> (Swagger : <https://localhost:7123/swagger>).

subsection[3. Démarrage du Simulateur d'Usine]3. Démarrage du Simulateur d'Usine
Ouvrir un troisième terminal et exécuter :

```
1 cd "Digital twin (projet OCP)/backend/DigitalTwin.Simulator"
2 dotnet run
```

Le simulateur injecte immédiatement les flux de télémétrie et rejoue le cycle de vie nominal et d'incidents sur les équipements.

subsection[4. Exécution des Tests Automatisés]4. Exécution des Tests Automatisés
Pour exécuter la suite des 83 tests unitaires du backend :

```
1 cd "Digital twin (projet OCP)/backend"
2 dotnet test
```